LAB 3: Motion Planning using ROS

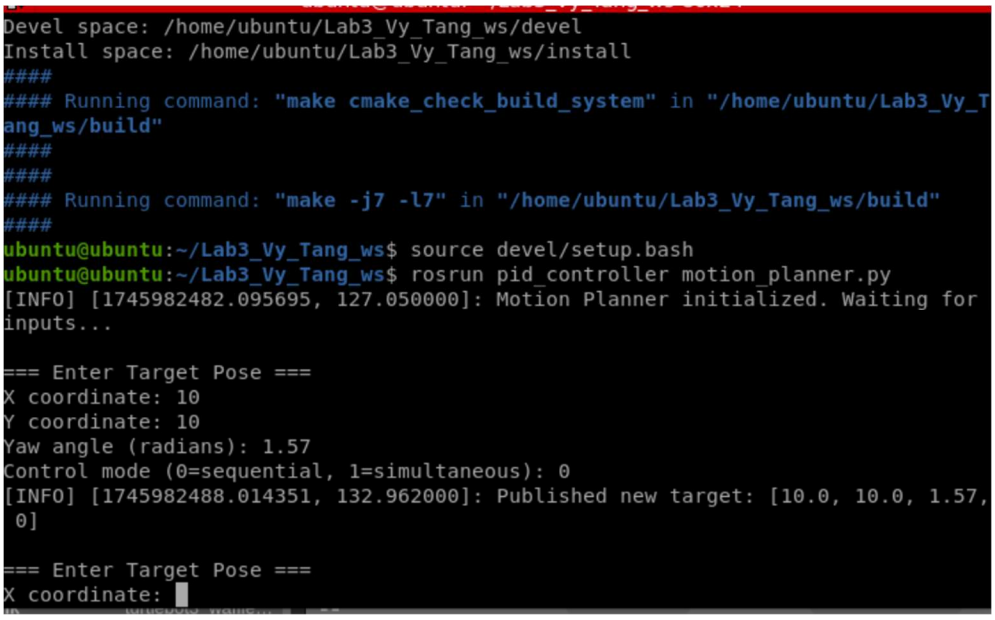
By: Vy Tang

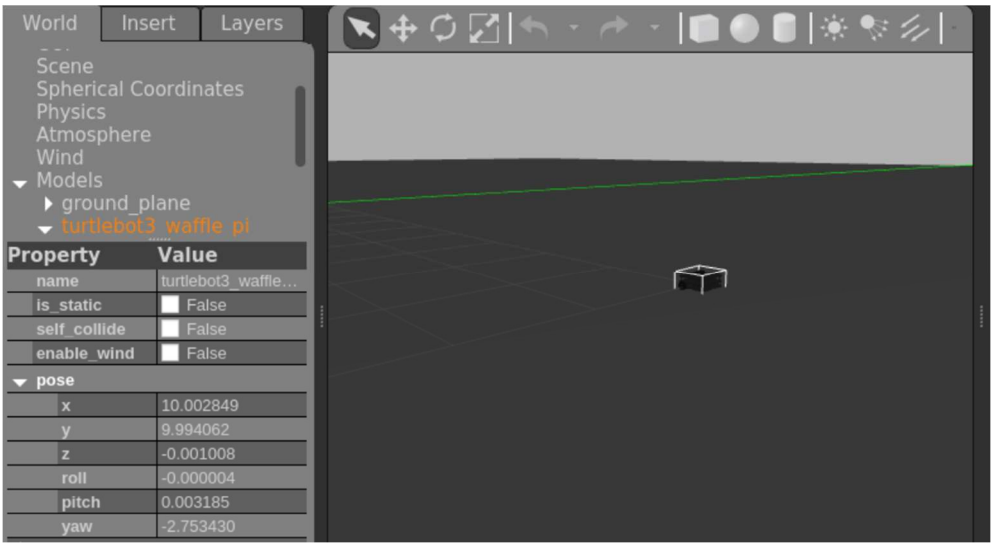
ID: 77659006

Objective

The objective of this lab report is to design and implement a PID controller to navigate the robot from an initial pose to the target poser from the user input. In the PID controller files, the PID gains were tuned to minimize errors and variations. The Gazebo software was used to simulate the differential drive robot to check the trajectory of the robot after user input.

Results





```
=== Enter Target Pose ===
X coordinate: 10
Y coordinate: 10
Yaw angle (radians): 1.57
Control mode (0=sequential, 1=simultaneous): 0
[INFO] [1745990011.905230, 5942.583000]: Published new target: [10.0, 10.0, 1.57
, 0]

=== Enter Target Pose ===
X coordinate: 15
Y coordinate: 20
Yaw angle (radians): 1.57
Control mode (0=sequential, 1=simultaneous): 0
[INFO] [1745990182.823946, 6107.678000]: Published new target: [15.0, 20.0, 1.57
, 0]
```

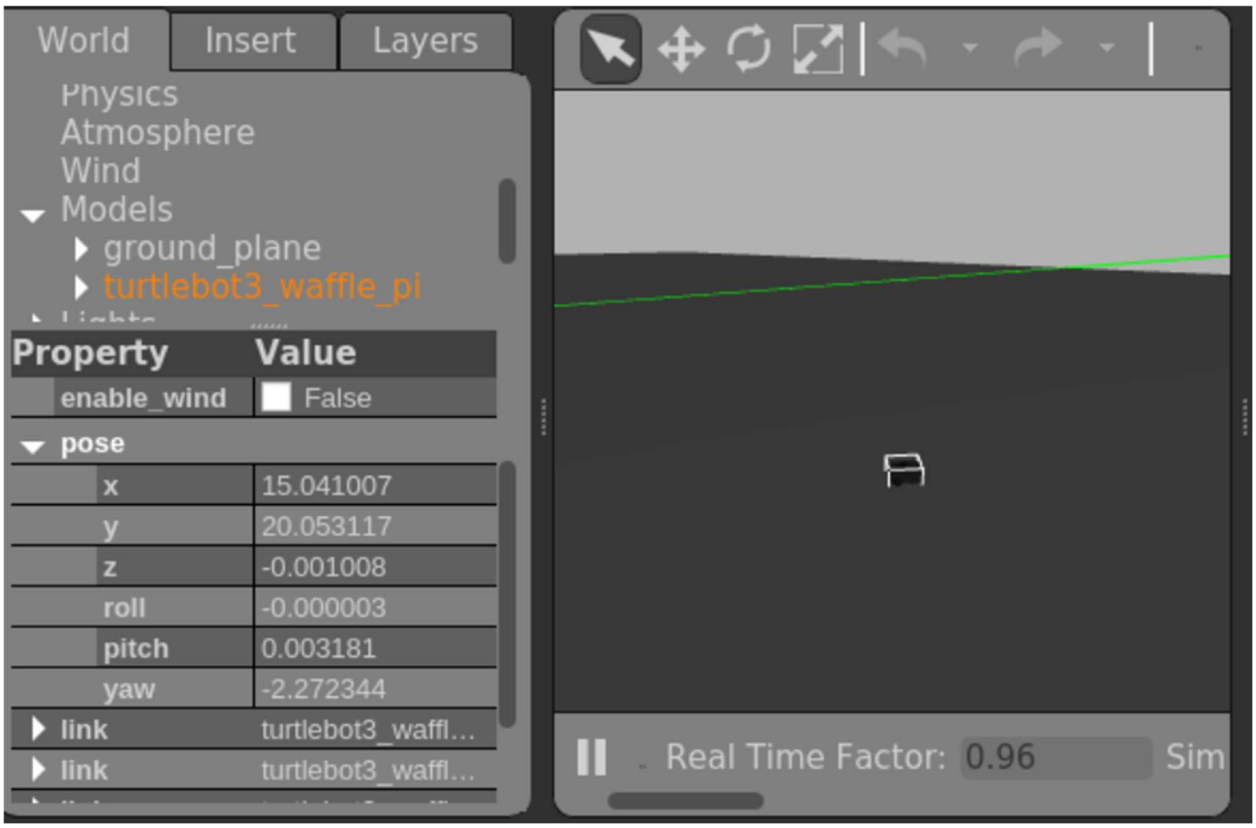
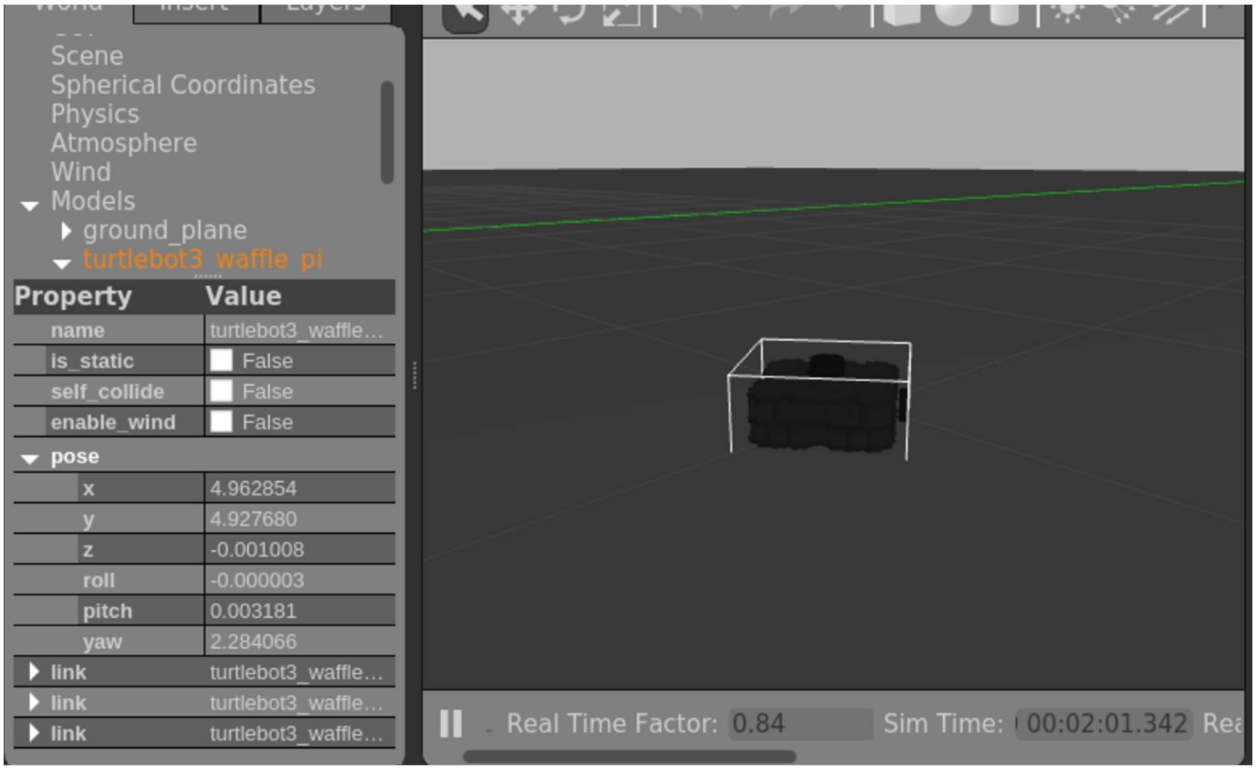


Fig 1. User input mode 0




```
ubuntu@ubuntu: ~/Lab3_Vy_Tang_ws 80x24
Build space: /home/ubuntu/Lab3_Vy_Tang_ws/build
Devel space: /home/ubuntu/Lab3_Vy_Tang_ws/devel
Install space: /home/ubuntu/Lab3_Vy_Tang_ws/install
####
#### Running command: "make cmake_check_build_system" in "/home/ubuntu/Lab3_Vy_T
ang_ws/build"
####
####
#### Running command: "make -j7 -l7" in "/home/ubuntu/Lab3_Vy_Tang_ws/build"
####
ubuntu@ubuntu:~/Lab3_Vy_Tang_ws$ source devel/setup.bash
ubuntu@ubuntu:~/Lab3_Vy_Tang_ws$ rosrun pid_controller motion_planner.py
[INFO] [1745983925.938683, 70.613000]: Motion Planner initialized. Waiting for i
nputs...

=== Enter Target Pose ===
X coordinate: 5
Y coordinate: 5
Yaw angle (radians): 1.57
Control mode (0=sequential, 1=simultaneous): 1
[INFO] [1745983934.022548, 77.775000]: Published new target: [5.0, 5.0, 1.57, 1]

=== Enter Target Pose ===
X coordinate:
```

```
=== Enter Target Pose ===
X coordinate: 5
Y coordinate: 5
Yaw angle (radians): 1.57
Control mode (0=sequential, 1=simultaneous): 1
[INFO] [1745990565.210300, 6472.621000]: Published new target: [5.0, 5.0, 1.57,
1]

=== Enter Target Pose ===
X coordinate: -5
Y coordinate: -5
Yaw angle (radians): -1.57
Control mode (0=sequential, 1=simultaneous): 1
[INFO] [1745990681.182358, 6582.450000]: Published new target: [-5.0, -5.0, -1.5
7, 1]
```

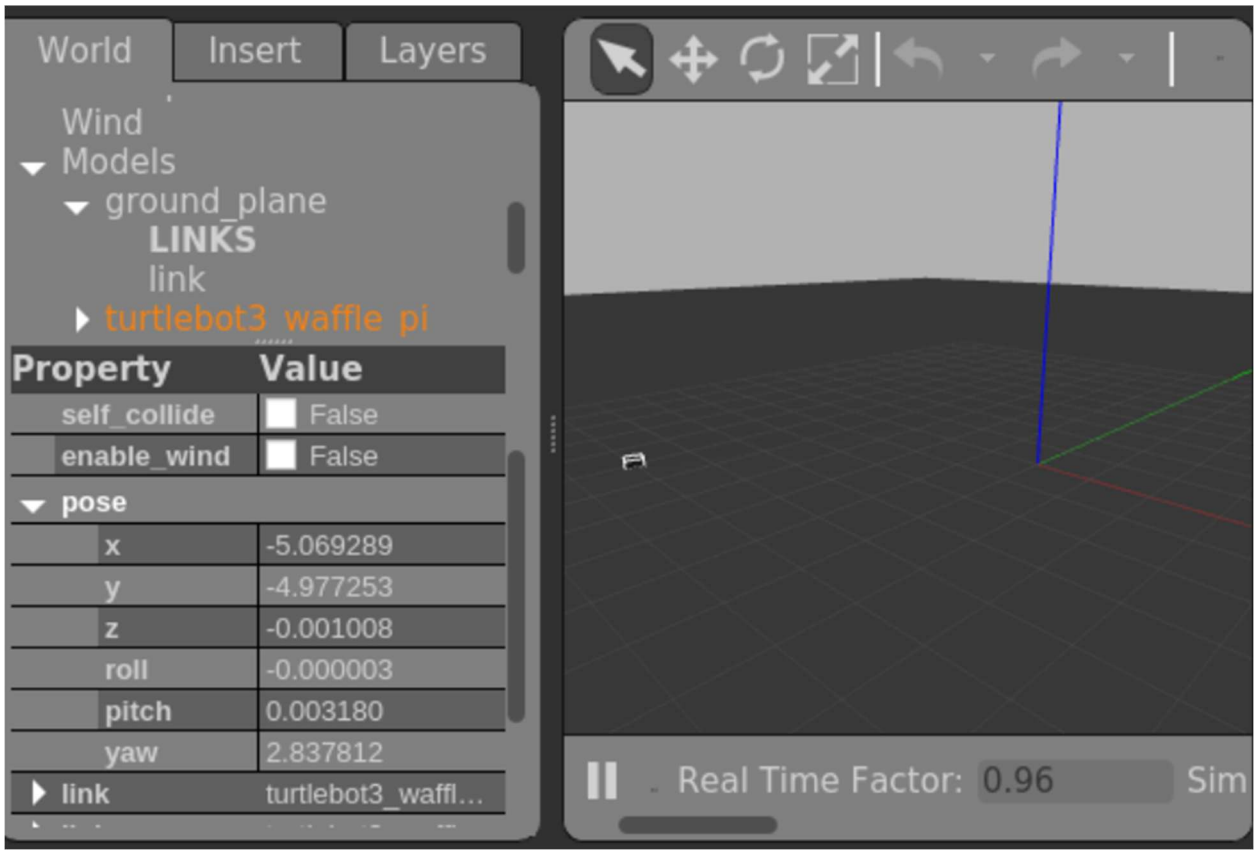


Fig 2. User input with mode 2

Code explanation

Pid_controller.py

```python
#!/usr/bin/env python3
import rospy
from geometry_msgs.msg import Twist
from nav_msgs.msg import Odometry
from std_msgs.msg import Float64MultiArray
import math

class PIDController:
    def __init__(self):
        # Initialize node and setup publishers/subscribers
        rospy.init_node('pid_controller')

        # ROS communication setup
        self.cmd_vel_pub = rospy.Publisher('/cmd_vel', Twist, queue_size=10)
        rospy.Subscriber('/reference_pose', Float64MultiArray, self.ref_callback)
        rospy.Subscriber('/odom', Odometry, self.odom_callback)

        # Control parameters
        self.rate = rospy.Rate(10)  # 10 Hz control loop
        self.last_time = rospy.Time.now()

        # PID parameters - now organized in dictionaries for easier management
        self.pid_params = {
            'linear': {
                'kp': 2.0,
                'ki': 0.01,
                'kd': 0.4,
                'max_output': 0.7,
                'min_output': -0.7,
                'integral_limit': 1.0
            },
            'angular': {
                'kp': 4.0,
                'ki': 0.01,
                'kd': 0.6,
                'max_output': 1.5,
                'min_output': -1.5,
                'integral_limit': 1.0
            }
        }

        # State variables
        self.ref_pose = {'x': 0, 'y': 0, 'theta': 0, 'mode': 0}  # target pose
        self.curr_pose = {'x': 0, 'y': 0, 'theta': 0}  # current pose

        # PID control variables
        self.prev_errors = {'linear': 0, 'angular': 0}
        self.integral_errors = {'linear': 0, 'angular': 0}
```

```python
    def ref_callback(self, msg):
        """Callback for reference pose updates"""
        self.ref_pose = {
            'x': msg.data[0],
            'y': msg.data[1],
            'theta': msg.data[2],
            'mode': msg.data[3]
        }

    def odom_callback(self, msg):
        """Callback for odometry updates"""
        pos = msg.pose.pose.position
        ori = msg.pose.pose.orientation
        yaw = self.quaternion_to_yaw(ori.x, ori.y, ori.z, ori.w)

        self.curr_pose = {
            'x': pos.x,
            'y': pos.y,
            'theta': yaw
        }

    @staticmethod
    def quaternion_to_yaw(x, y, z, w):
        """Convert quaternion orientation to yaw angle"""
        siny_cosp = 2 * (w * z + x * y)
        cosy_cosp = 1 - 2 * (y * y + z * z)
        return math.atan2(siny_cosp, cosy_cosp)

    def calculate_errors(self):
        """Calculate current position and orientation errors"""
        # Position errors
        error_x = self.ref_pose['x'] - self.curr_pose['x']
        error_y = self.ref_pose['y'] - self.curr_pose['y']

        # Distance and angle to target
        distance = math.hypot(error_x, error_y)
        target_angle = math.atan2(error_y, error_x)

        # Angular error (normalized to [-pi, pi])
        error_theta = self.normalize_angle(target_angle - self.curr_pose['theta'])

        return distance, error_theta
```

```python
    @staticmethod
    def normalize_angle(angle):
        """Normalize angle to [-pi, pi]"""
        return math.atan2(math.sin(angle), math.cos(angle))

    def compute_pid(self, error, prev_error, integral_error, pid_type):
        """Compute PID output for given error and PID type"""
        params = self.pid_params[pid_type]

        # Calculate time step
        current_time = rospy.Time.now()
        dt = (current_time - self.last_time).to_sec()
        self.last_time = current_time

        # Proportional term
        p = params['kp'] * error

        # Integral term with anti-windup
        self.integral_errors[pid_type] += error * dt
        self.integral_errors[pid_type] = max(-params['integral_limit'], min(params['integral_limit'], self.integral_errors[pid_type]))
        i = params['ki'] * self.integral_errors[pid_type]

        # Derivative term
        if dt > 0:
            d = params['kd'] * (error - prev_error) / dt
        else:
            d = 0

        # Compute output and apply limits
        output = p + i + d
        output = max(params['min_output'], min(params['max_output'], output))

        # Apply deadzone for very small outputs
        if abs(output) < 0.05:
            output = 0.0

        return output

    def control_sequence_mode0(self, distance, error_theta):
        """Control sequence for mode 0 (rotate then move)"""
        cmd = Twist()
        final_error = self.normalize_angle(self.ref_pose['theta'] - self.curr_pose['theta'])

        if abs(error_theta) > 0.05:
            # Rotate to face target
            cmd.angular.z = self.compute_pid(error_theta, self.prev_errors['angular'], self.integral_errors['angular'], 'angular')
        elif distance > 0.1:
            # Move toward target
            cmd.linear.x = self.compute_pid(distance, self.prev_errors['linear'], self.integral_errors['linear'], 'linear')
        else:
            # Final orientation adjustment
            if abs(final_error) > 0.05:
                cmd.angular.z = self.compute_pid(final_error, self.prev_errors['angular'], self.integral_errors['angular'], 'angular')
            else:
                # Reset integral terms when target is reached
                self.integral_errors = {'linear': 0, 'angular': 0}

        return cmd

    def control_sequence_mode1(self, distance, error_theta):
        """Control sequence for mode 1 (rotate and move simultaneously)"""
        cmd = Twist()
        final_error = self.normalize_angle(self.ref_pose['theta'] - self.curr_pose['theta'])

        if distance > 0.1:
            # Combined movement and rotation
            cmd.linear.x = self.compute_pid(distance, self.prev_errors['linear'], self.integral_errors['linear'], 'linear')
            cmd.angular.z = self.compute_pid(error_theta, self.prev_errors['angular'], self.integral_errors['angular'], 'angular')

            # Apply scaling factors based on conditions
            angle_penalty = max(0.0, 1.0 - abs(error_theta))
            cmd.linear.x *= angle_penalty

            if distance < 0.5:
                cmd.linear.x *= distance / 0.5

            if abs(error_theta) < 0.5:
                cmd.angular.z *= abs(error_theta) / 0.5
        else:
            # Final orientation adjustment
            if abs(final_error) > 0.05:
                cmd.angular.z = self.compute_pid(final_error, self.prev_errors['angular'], self.integral_errors['angular'], 'angular')
            else:
                # Reset integral terms when target is reached
                self.integral_errors = {'linear': 0, 'angular': 0}

        return cmd
```

```python
    def control(self):
        """Main control function"""
        # Calculate current errors
        distance, error_theta = self.calculate_errors()

        # Select control mode
        if self.ref_pose['mode'] == 0:
            cmd = self.control_sequence_mode0(distance, error_theta)
        else:
            cmd = self.control_sequence_mode1(distance, error_theta)

        # Publish command
        self.cmd_vel_pub.publish(cmd)

        # Update previous errors
        self.prev_errors = {
            'linear': distance,
            'angular': error_theta
        }

    def run(self):
        """Main loop"""
        while not rospy.is_shutdown():
            self.control()
            self.rate.sleep()

if __name__ == "__main__":
    try:
        controller = PIDController()
        controller.run()
    except rospy.ROSInterruptException:
        pass
```

Firstly, we initialize the ROS node named pid_controller with velocity command publisher and subscribers i.e. reference_node and odometry. Then we will also initialize the PID gains for linear/angular control and initialize variables for target pose and current pose.

Then, there are the callback functions where the reference callback will update target pose, and the odometry callback will convert odometry quarternions (vector) into yaw and updates current pose. Next, I will compute the distance and angle errors to target using hypot() and atan2(). After that, I will normalize angular errors.

The next function is the PID mathematical logic under the compute_pid function. In this function, each term p, i, d is computed separately and then added together in the end using output variable. I also set a limit where if the output is not to a certain threshold of 0.05, the output will be automatically set to zero.

After this, there will be a function that controls the modes. The sequential mode, mode 0, will rotate the robot to face the target and will start moving towards that direction. It will then adjust orientation and repeat. For simultaneous mode (mode 1), the robot will move and turn at the same time.

The main loop will then execute all these functions where it will calculate errors, selects the control mode, and publishes the Twist command as well as provides updates and error history for the robot to fix its trajectory to its goal node.

Motion_planner.py

```python
#!/usr/bin/env python3
import rospy
import math
from std_msgs.msg import Float64MultiArray
from nav_msgs.msg import Odometry

class MotionPlanner:
    def __init__(self):
        rospy.init_node('motion_planner')

        # Publishers and Subscribers
        self.pose_publisher = rospy.Publisher('/reference_pose', Float64MultiArray, queue_size=10)
        rospy.Subscriber('/odom', Odometry, self.odom_callback)

        # State variables
        self.current_pose = {'x': 0.0, 'y': 0.0, 'yaw': 0.0}
        self.target_pose = {'x': 0.0, 'y': 0.0, 'yaw': 0.0, 'mode': 0}
        self.goal_reached = True
        self.rate = rospy.Rate(10)  # 10Hz control loop

    def odom_callback(self, msg):
        """Callback for Odometry messages to update current pose"""
        self.current_pose['x'] = msg.pose.pose.position.x
        self.current_pose['y'] = msg.pose.pose.position.y
        self.current_pose['yaw'] = self.quaternion_to_yaw(msg.pose.pose.orientation)

    @staticmethod
    def quaternion_to_yaw(orientation):
        """Convert quaternion to yaw angle (radians)"""
        siny_cosp = 2 * (orientation.w * orientation.z + orientation.x * orientation.y)
        cosy_cosp = 1 - 2 * (orientation.y**2 + orientation.z**2)
        return math.atan2(siny_cosp, cosy_cosp)

    def get_user_input(self):
        """Prompt user for target pose and mode with validation"""
        while True:
            try:
                print("\n=== Enter Target Pose ===")
                x = float(input("X coordinate: "))
                y = float(input("Y coordinate: "))
                yaw = float(input("Yaw angle (radians): "))
                mode = int(input("Control mode (0=sequential, 1=simultaneous): "))

                if mode not in (0, 1):
                    raise ValueError("Mode must be 0 or 1")
                return x, y, yaw, mode
            except ValueError as e:
                rospy.logwarn(f"Invalid input: {e}. Please try again.")

    def check_goal_reached(self, tolerance=0.1):
        """Check if current pose is within tolerance of target"""
        dx = self.target_pose['x'] - self.current_pose['x']
        dy = self.target_pose['y'] - self.current_pose['y']
        return math.hypot(dx, dy) < tolerance

    def run(self):
        """Main control loop"""
        while not rospy.is_shutdown():
            if self.goal_reached:
                # Get new target from user
                x, y, yaw, mode = self.get_user_input()
                self.target_pose.update({'x': x, 'y': y, 'yaw': yaw, 'mode': mode})

                # Publish target
                target_msg = Float64MultiArray()
                target_msg.data = [x, y, yaw, mode]
                self.pose_publisher.publish(target_msg)
                rospy.loginfo(f"Published new target: {target_msg.data}")
                self.goal_reached = False

            # Check goal status
            self.goal_reached = self.check_goal_reached()
            self.rate.sleep()

if __name__ == "__main__":
    try:
        planner = MotionPlanner()
        rospy.loginfo("Motion Planner initialized. Waiting for inputs...")
        planner.run()
    except rospy.ROSInterruptException:
        pass
```

The first part is initializing the node motion_planner with a publisher for reference_pose and a subscriber to odometry. There also variables that track current and target poses and goal status.

Then, the odometry callback function will extract the (x,y) position and converts the robot's orientation to yaw using quarternion math. Then, there will be a function that prompts the user to get (x, y) coordinates, yaw angle, and control mode.

After getting the user input, it will check if the robot have reached that goal using the Euclidean distance to target and returns true if the robot is within 0.1 units away from the user input target. When the goal is finally reached, the program will prompt another user input and the whole program will be reran again.